



DÉTECTER DES FAUX BILLETS AVEC PYTHON

SOMMAIRE

- **Réaliser une analyse exploratoire des billets**
- **Détecter les faux billets en utilisant plusieurs algorithmes**
- **Evaluer les algorithmes en utilisant la matrice de confusion**
- **Création d'une fonction de détection des faux billets**



CONTEXTE ET OBJECTIF

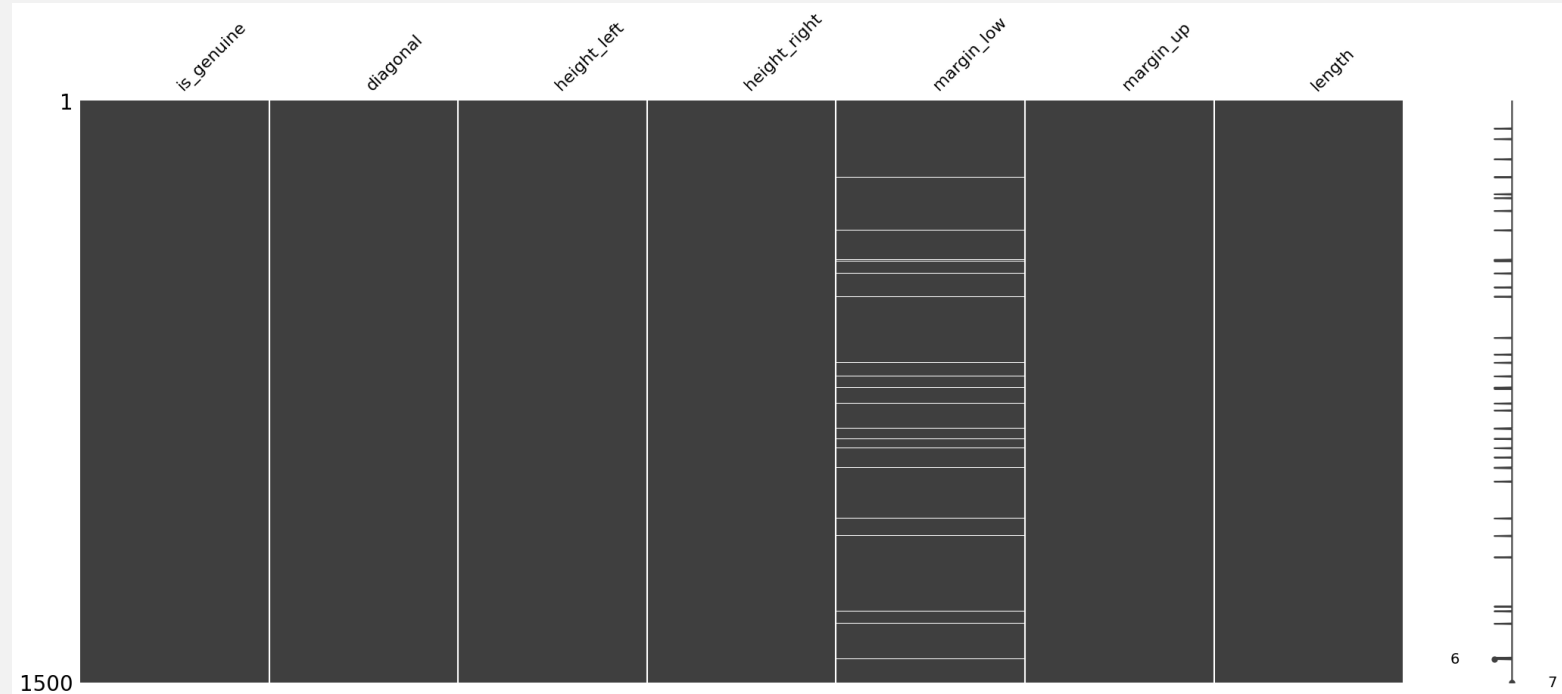


- Lutter contre la contrefaçon des faux billets
- Mettre à disposition des équipes une application de détection

JEU DE DONNÉES ET VARIABLE CIBLE



- Colonnes : is_genuine, length, margin_up, height_left, height_right, diagonal,
- cible margin_low
- Taille : 1500 observations.



STATISTIQUES DESCRIPTIVES



```

count    diagonal  height_left  height_right  margin_low  margin_up \
mean    171.958440  104.029533  103.920307    4.485967    3.151473
std      0.305195    0.299462    0.325627    0.663813    0.231813
min     171.040000  103.140000  102.820000    2.980000    2.270000
25%     171.750000  103.820000  103.710000    4.015000    2.990000
50%     171.960000  104.040000  103.920000    4.310000    3.140000
75%     172.170000  104.230000  104.150000    4.870000    3.310000
max     173.010000  104.880000  104.950000    6.900000    3.910000

count    length
mean     112.67850
std       0.87273
min      109.49000
25%      112.03000
50%      112.96000
75%      113.34000
max      114.44000
NaN par colonne:
  is_genuine    0
diagonal        0
height_left     0
height_right    0
margin_low      37
margin_up       0
length         0
dtype: int64

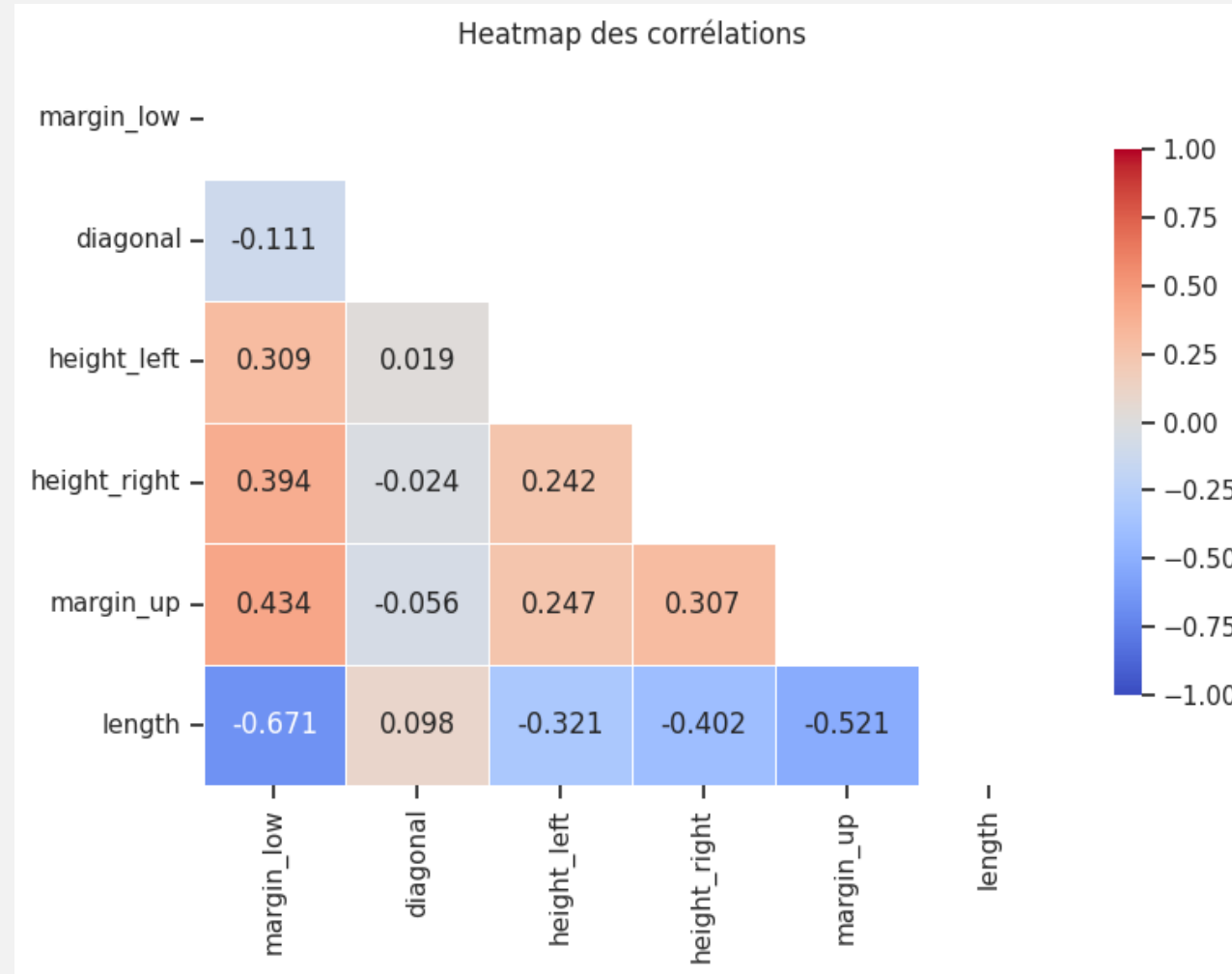
```

```

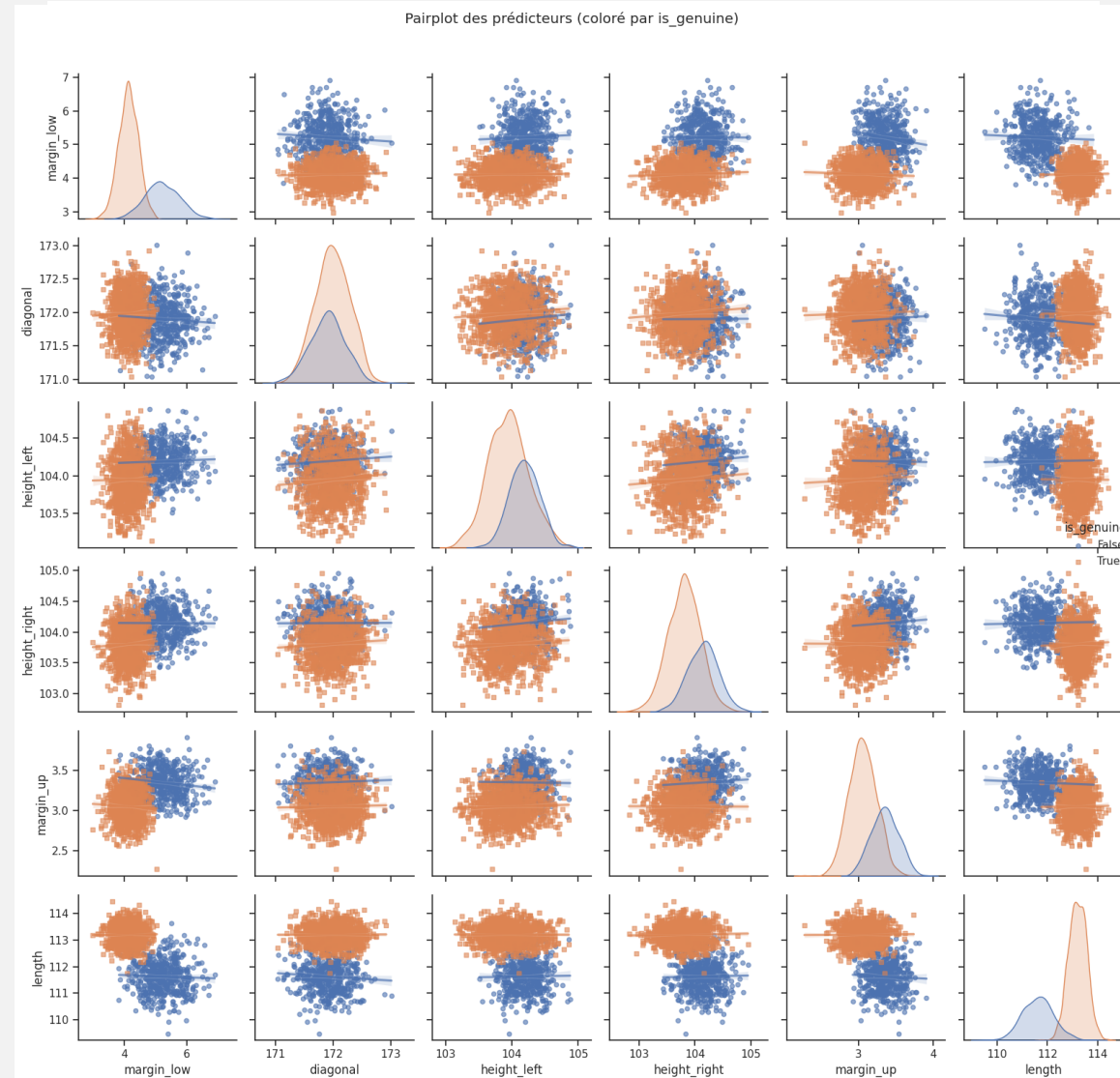
... Variances:
     length      0.761657
margin_up      0.053737
height_left    0.089677
height_right   0.106033
diagonal       0.093144
dtype: float64
Nunique par colonne:
     length      336
margin_up      123
height_left    155
height_right   170
diagonal       159
dtype: int64
Corr matrix:
      length  margin_up  height_left  height_right  diagonal
length      1.000    -0.521    -0.321    -0.402    0.098
margin_up   -0.521    1.000     0.247     0.307   -0.056
height_left -0.321    0.247     1.000     0.242    0.019
height_right -0.402    0.307     0.242     1.000   -0.024
diagonal     0.098   -0.056     0.019    -0.024    1.000
Condition number: 4.6585053358668125
VIF (statsmodels):
     length      22676.458356
margin_up      260.785987
height_left    112879.408271
height_right   100054.247709
diagonal       169341.929149
dtype: float64
length : R2 = 0.364984 -> VIF = 1.574765
margin_up : R2 = 0.287504 -> VIF = 1.403517
height_left : R2 = 0.126863 -> VIF = 1.145295
height_right : R2 = 0.186504 -> VIF = 1.229263
diagonal : R2 = 0.012629 -> VIF = 1.012790

```

CORRÉLATIONS ENTRE PRÉDICTEURS



PAIRPLOT DES PREDICTEURS

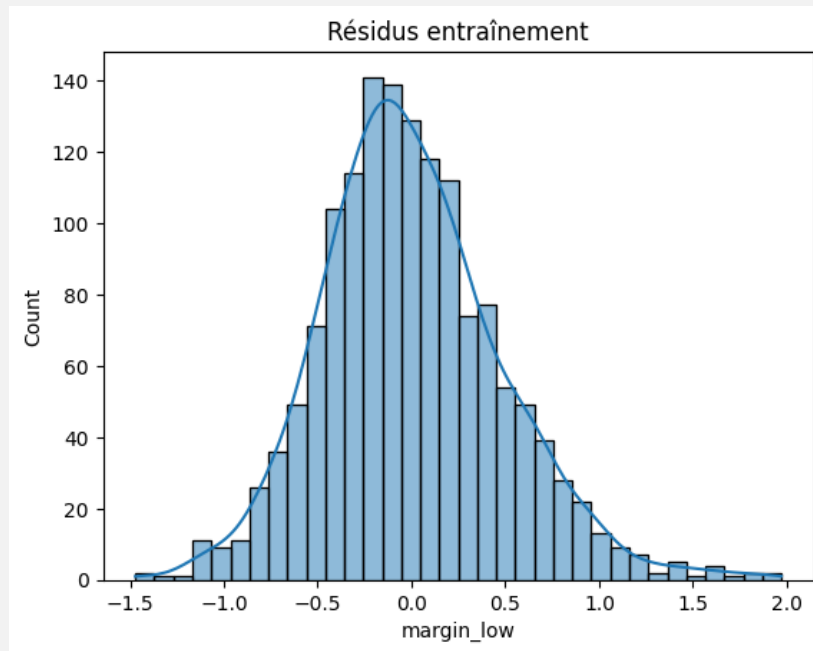


IMPUTATION DE LA COLONNE MARGIN_LOW

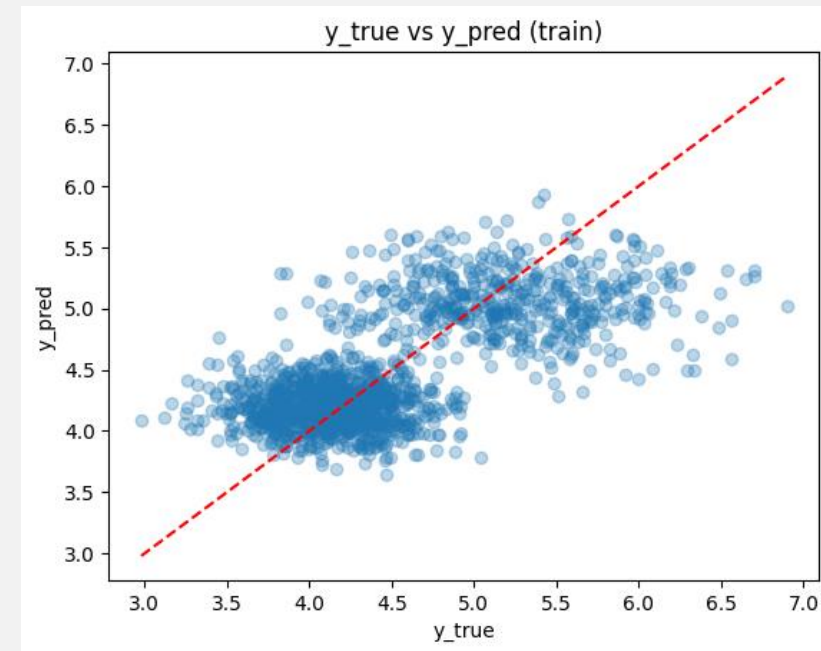
- Approche : régression linéaire (prédiction à base des autres colonne)

RÉGRESSION LINÉAIRE

Distribution normale des résidus



Prédiction de margin_low



DIAGNOSTICS RL APPROFONDIS

- Résidus : distribution centrée, dispersion visible.
- ~
- Action : transformer variables ou utiliser modèles non linéaires

- **Résultats** : Validation $RMSE = 0.4768$, $R^2 = 0.4837$.

DETECTION DES FAUX BILLETS



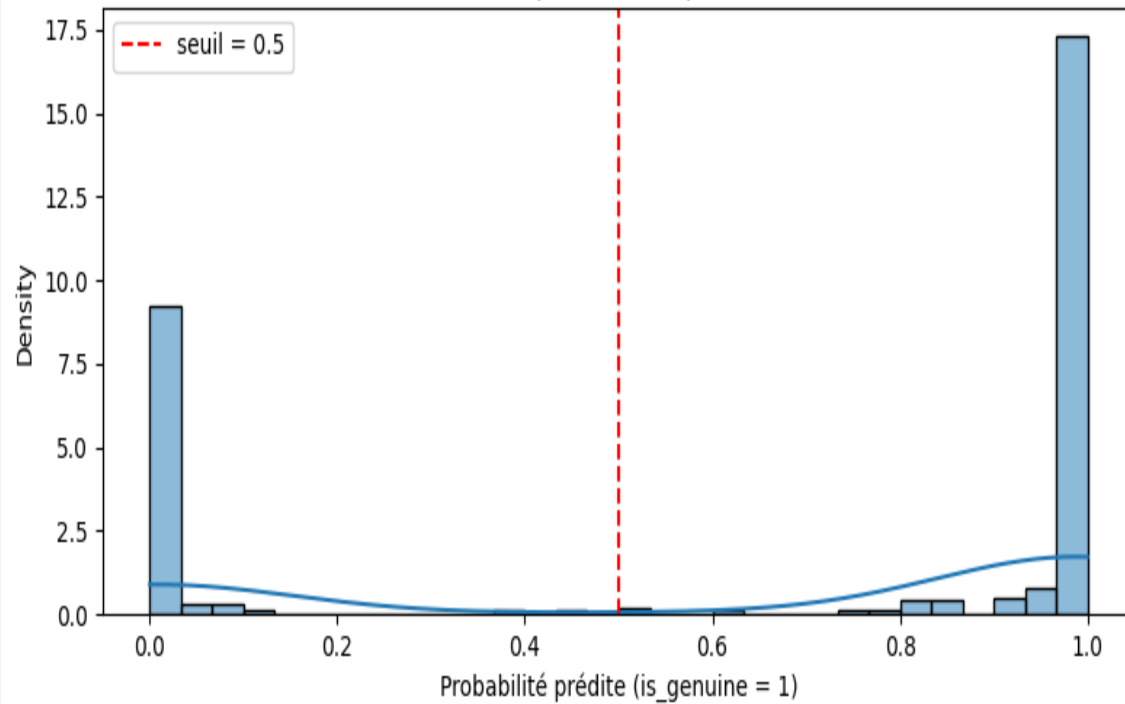
- REGRESSION LOGISTIQUE
- KNN
- RANDOM FOREST
- K-MEANS (non-supervisé)

PIPELINE ET CROSS-VALIDATION

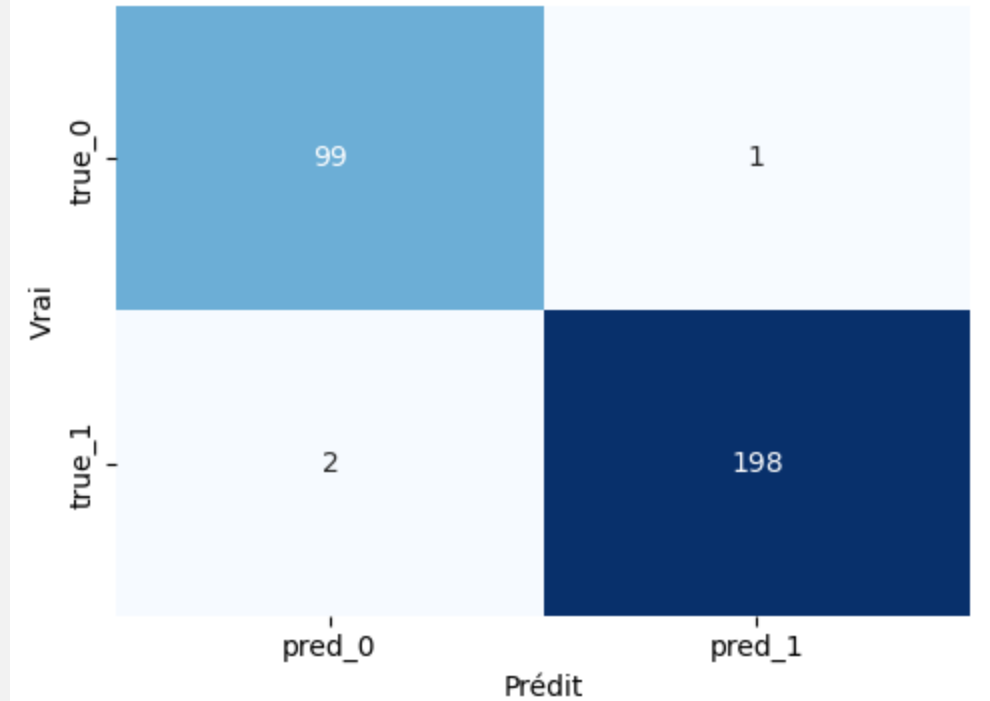
- Pipeline : préproc (impute, scaler, encodage) → modèle.
- CV : K-fold (même split pour comparaisons).
- Reproductibilité : `random_state` fixé, `joblib.dump` du pipeline.

RÉGRESSION LOGISTIQUE

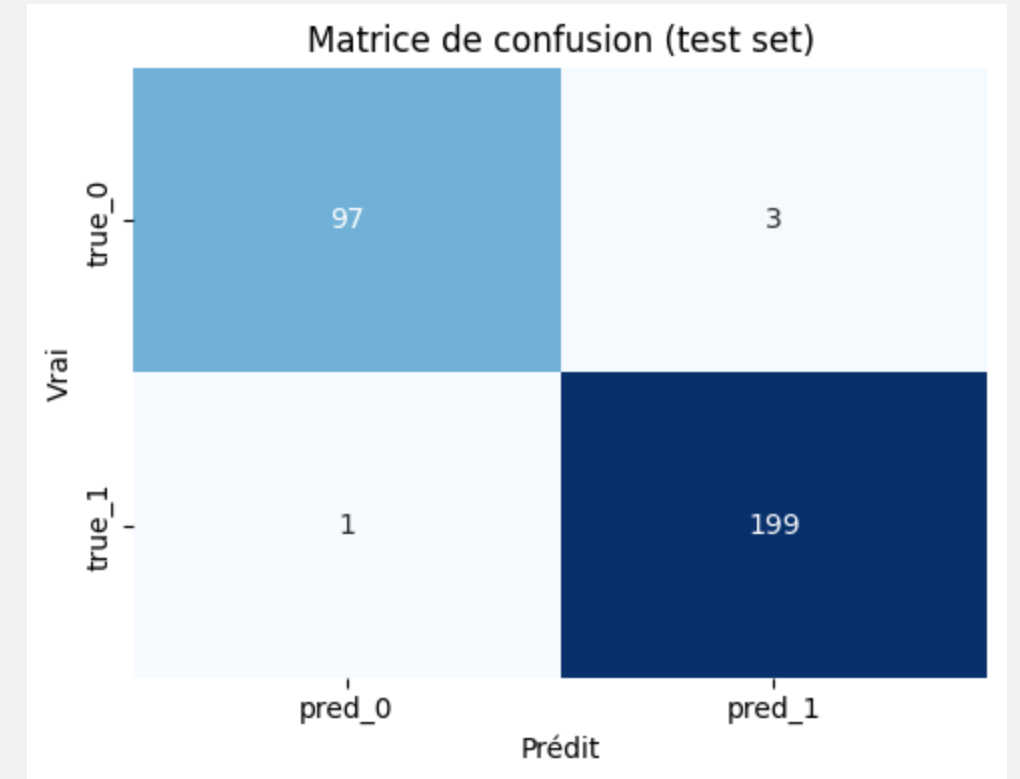
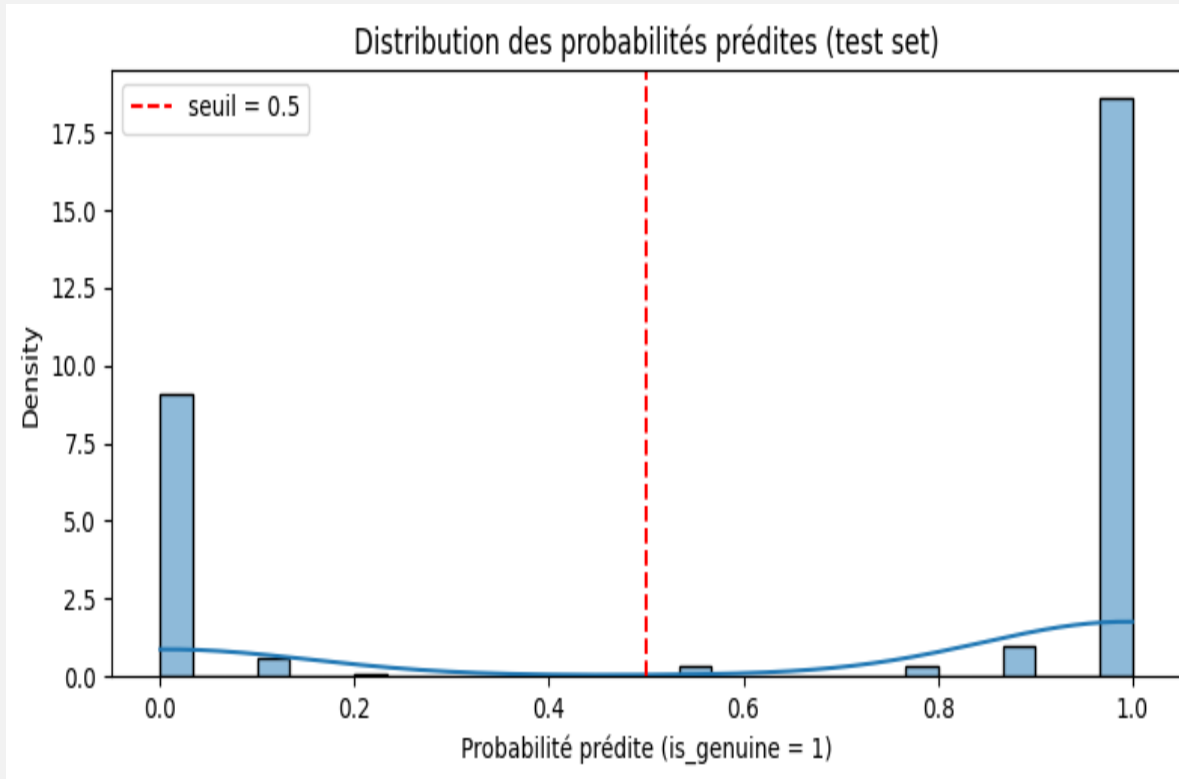
Distribution des probabilités prédites (test set)



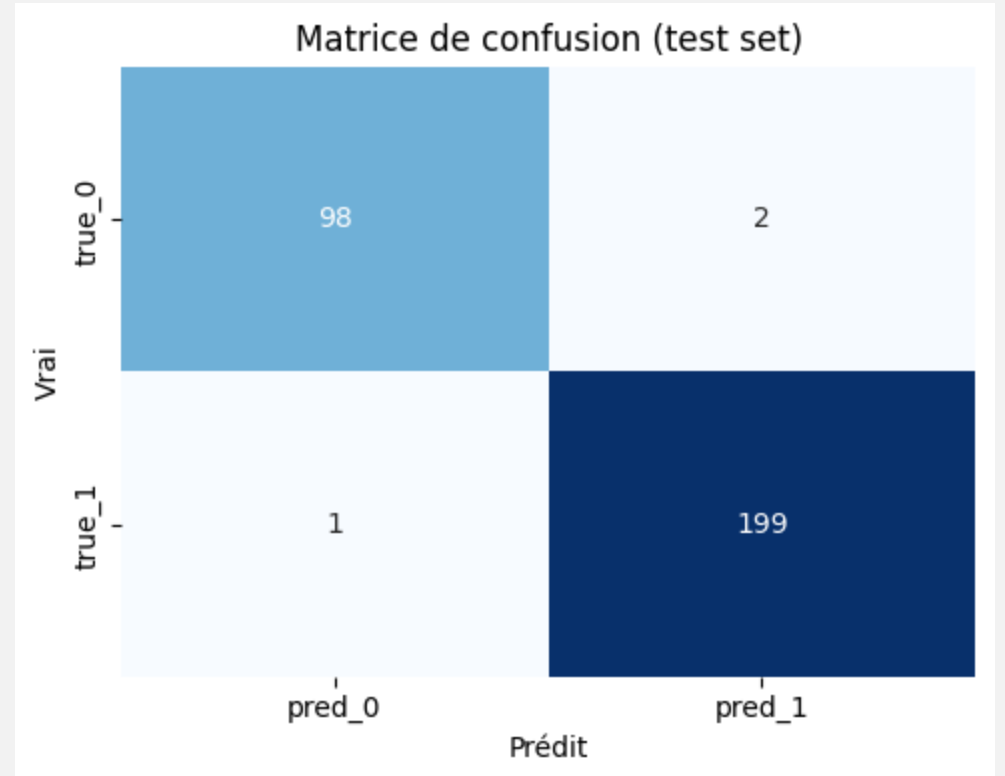
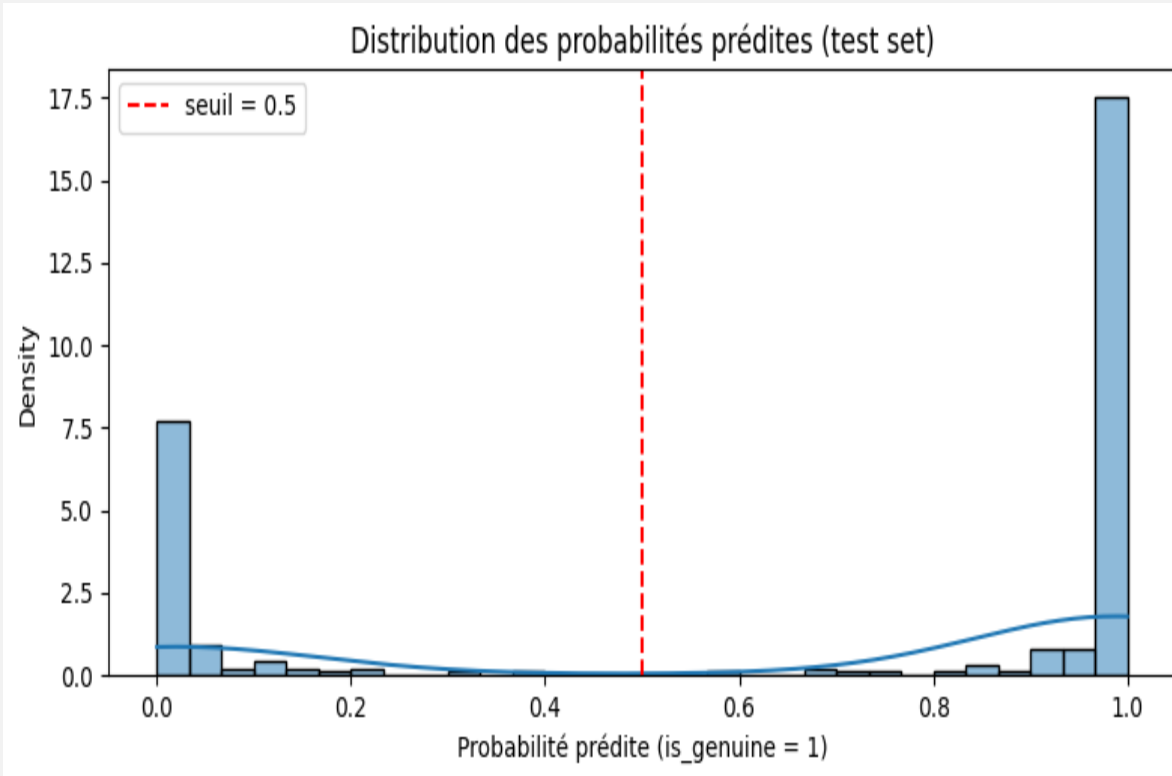
Matrice de confusion (test set)



K-NEAREST NEIGHBORS (KNN)

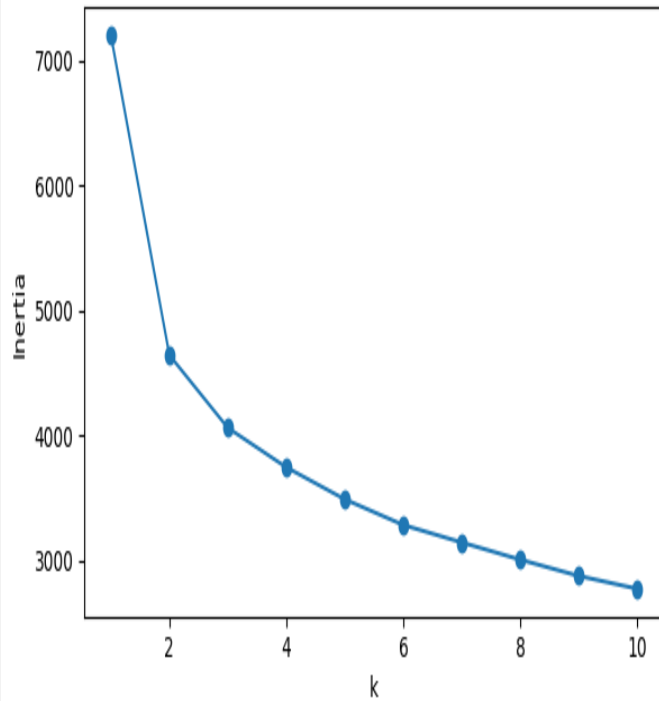


RANDOM FOREST

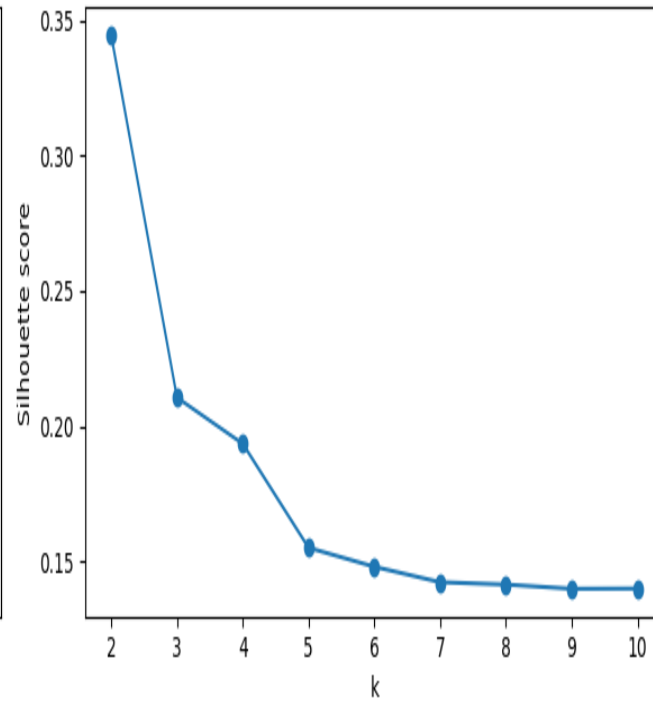


K-MEANS

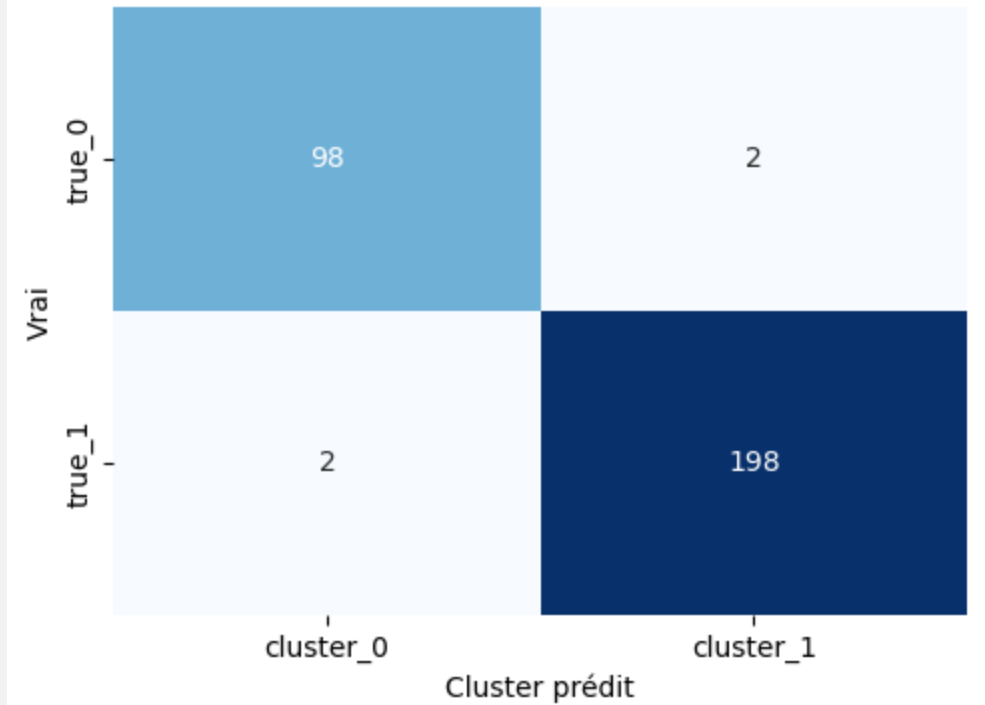
Elbow method



Silhouette score



Confusion entre vraie classe et cluster (test set)



COMPARAISON SYNTHÉTIQUE DES MODÈLES

Model	Accuracy	AUC	Precision	Recall	F1	Notes
Régression logistique	0.9900	0.9995	0.9950	0.9900	0.9925	Très bonne performance; très interprétable
KNN	0.9867	0.9971	0.9851	0.9950	0.9900	Performant; sensible au scaling et coût en production
Random Forest	0.9900	0.9992	0.9900	0.9950	0.9925	Robuste, bonne importance de features, bon compromis perf/robustesse
K-Means (k=2)	—	—	—	—	—	ARI=0.9466; Homogeneity=0.8900; Completeness=0.8900 — segmentation nette et utile comme feature

RECOMMANDATIONS OPÉRATIONNELLES ET PLAN D'ACTION — PILOTE DÉTECTION FAUX BILLETS 1/3

- **Monitoring et gouvernance**
- **Suivi quotidien** : taux faux billet déclaré vrai, taux vrai billet déclaré faux, volume d'alertes, temps moyen de vérification.
- **Collecte et annotation** : enregistrer tous les cas mal classés pour ré-entraînement.



RECOMMANDATIONS OPÉRATIONNELLES ET PLAN D'ACTION — PILOTE DÉTECTION FAUX BILLETS 2/3

- **Actions immédiates**
- **Lancer pilote contrôlé** : 1–2 sites pour valider en conditions réelles.
- **Workflow opérationnel** : toute prédiction = *faux* → **vérification humaine rapide** + **logging** des cas.
- **Calculer coût attendu** pour choisir le seuil optimal : $\hat{Coût\ attendu} = FN \times \text{\text{€}}/FN + FP \times \text{\text{€}}/FP$





RECOMMANDATIONS OPÉRATIONNELLES ET PLAN D'ACTION — PILOTE DÉTECTION FAUX BILLETS 3/3

- **Plan 90 jours**
- **J0–J30** : déploiement pilote, instrumentation, collecte d'erreurs.
- **J30–J60** : analyse qualitative des erreurs, ajustement des seuils, tests de robustesse.
- **J60–J90** : extension progressive, automatisation du monitoring, décision de mise en production



CONCLUSION ET REMERCIEMENTS



MERCI POUR VOTRE ÉCOUTE ET
VOTRE ATTENTION